GUIA RÁPIDO · QUARKUS

# Criando um projeto Quarkus no VSCode

Passo a passo para gerar um novo projeto Quarkus usando a extensão oficial do VSCode, do zero até a primeira execução em modo dev.

`modelo didático baseado em pw2.rpmhub.dev`

---

## 1 Instalar a extensão do Quarkus

No VSCode, abra a aba de extensões e procure pela extensão oficial da Red Hat.

| ONDE | COMO |
|---|---|
| Atalho | `Ctrl+Shift+X` |
| Buscar por | `Quarkus` |
| Instalar | **Quarkus Tools** (publisher: Red Hat) |

---

## 2 Abrir a paleta de comandos

É por ela que se acessa qualquer comando da extensão.

```
Ctrl + Shift + P
```

---

## 3 Executar o comando de criação

Digite o nome do comando e selecione-o na lista:

```
Quarkus: Create a Quarkus Project
```

---

## 4 Responder ao assistente (wizard)

O VSCode vai perguntar, em sequência, os dados do projeto:

| PERGUNTA | O QUE ESCOLHER |
|---|---|
| Gerenciador de build | `MAVEN` |
| Group Id | `org.acme` |
| Artifact Id | nome do projeto, ex: `quarkus-getting-started` |
| Versão | manter `1.0.0-SNAPSHOT` |
| Nome do pacote | `org.acme` |
| Versão do Quarkus | manter a sugerida (LTS) |

---

## 5 Selecionar as dependências (extensions)

Esta é a etapa mais importante: aqui você escolhe o que o projeto vai poder fazer. Para reproduzir os exercícios de REST, REST Client e JWT, marque:

- ☐ `RESTEasy Reactive` (quarkus-rest) — criar endpoints REST (recursos com @Path, @GET, etc.)
- ☐ `RESTEasy Reactive Jackson` (quarkus-rest-jackson) — conversão automática para/de JSON
- ☐ `REST Client Reactive` (quarkus-rest-client) — consumir APIs externas (interfaces com @RegisterRestClient)
- ☐ `REST Client Reactive Jackson` (quarkus-rest-client-jackson) — JSON também no lado cliente
- ☐ `SmallRye JWT` (quarkus-smallrye-jwt) — validar tokens recebidos (necessário para @RolesAllowed)
- ☐ `SmallRye JWT Build` (quarkus-smallrye-jwt-build) — gerar tokens dentro da própria aplicação

**Nota:** nas versões atuais do Quarkus (3.9+), as extensões REST reativas foram renomeadas — a busca no VSCode ainda aceita os nomes antigos, mas o `pom.xml` gerado usa os artifactIds novos mostrados entre parênteses acima. Funcionalmente é o mesmo conjunto de dependências.

**Dica:** não se preocupe em acertar tudo de uma vez — qualquer extensão pode ser adicionada depois editando o `pom.xml` ou rodando `./mvnw quarkus:add-extension -Dextensions="nome-da-extensao"`.

---

## 6 Escolher a pasta de destino

Selecione em qual pasta do seu sistema de arquivos o projeto será criado. O VSCode gera a estrutura completa (pom.xml, src/main/java, src/main/resources, etc.).

---

## 7 Abrir e rodar o projeto

O VSCode abre a pasta do projeto automaticamente. No terminal integrado (`Ctrl+`), rode:

```
./mvnw compile quarkus:dev
```

O modo dev fica escutando alterações no código e recarrega automaticamente. Acesse no navegador:

```
http://localhost:8080
```

**Bônus:** com o app rodando em modo dev, abra `http://localhost:8080/q/dev` para ver a Dev UI — um painel visual com todas as extensões instaladas, endpoints REST detectados e ferramentas de depuração.

---

**Alternativa sem VSCode: code.quarkus.io**

Se preferir não usar o wizard da extensão, acesse **code.quarkus.io**, marque as mesmas dependências do Passo 5, clique em "Generate your application" e baixe o `.zip`. Depois é só descompactar e abrir a pasta no VSCode — o restante (Passo 7 em diante) é idêntico.

---